

AT32 PMSM FOC Magnetic Encoder Quick Start Guide

Introduction

This document introduces how to use AT-MOTOR-EVB evaluation board equipped with AT32 motor control library to drive PMSM motor (with a magnetic encoder), and details debugging SOP. ArteryTek offers many motor development boards, and this document uses the AT-MOTOR-EVB as an example for illustration

Applicable products:

Part number	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32M412/M416 series

Contents

1	Motor control library algorithms	6
2	Environment requirements	7
2.1	Hardware requirements	7
2.2	Software requirements	8
3	Quick start guide	12
3.1	Modify motor control mode and parameters	13
3.2	Connection settings	18
3.3	Magnetic encoder SPI interface.....	18
3.4	Open loop control	19
3.5	Magnetic encoder zero-position calibration	21
3.6	Voltage control.....	22
3.7	Auto identification and tuning of parameters.....	23
3.8	ID tune.....	25
3.9	IQ tune	27
3.10	Current loop control	29
3.11	Speed loop control	31
3.12	Position loop control	32
3.13	Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM).....	35
3.14	MTPA/MTPV control (Interior Permanent Magnet Synchronous Motor, IPMSM)	37
3.15	External voltage control/software command control	41
3.16	Magnetic encoder offset angle calibration	43
4	Revision history.....	45

List of tables

Table 1. Correspondence between Flash size and ROM.....	8
Table 2. Macro definition setting (internal crystal)	13
Table 3. Macro definition setting (EVB version).....	13
Table 4. Macro definition setting (low-side MOS inverted output).....	13
Table 5. Macro definition setting (vector control mode).....	13
Table 6. Macro definition setting (current sampling method)	13
Table 7. Macro definition setting (two-shunt current sampling method).....	14
Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)	14
Table 9. Macro definition setting (dual ADCs current sampling)	14
Table 10. Macro definition setting (magnetic encoder).....	14
Table 11. Macro definition setting (encoder reverse counting).....	14
Table 12. Macro definition setting (speed measurement method)	14
Table 13. Macro definition setting (magnetic encoder deviation angle calibration)	15
Table 14. Macro definition setting (d/q-axis current low-pass filter)	15
Table 15. Macro definition setting (a/b/c phase current low-pass filter)	15
Table 16. Macro definition setting (motor type)	15
Table 17. Macro definition setting (MTPA/MTPV lookup table generation).....	15
Table 18. Macro definition setting (MTPA/MTPV lookup table-based torque control)	15
Table 19. Macro definition setting (field weakening control).....	16
Table 20. Macro definition setting (speed limit during torque control).....	16
Table 21. Macro definition setting (current decouple control).....	16
Table 22. Macro definition setting (motor winding parameter identification).....	16
Table 23. Macro definition setting (Artery motor UI tool)	16
Table 24. Motor parameter definition	17
Table 25. Document revision history.....	45

List of figures

Figure 1. AT-MOTOR-EVB V2.x	7
Figure 2. AT32F413RCT7 ROM settings (Keil)	9
Figure 3. AT32F413RCT7 ROM settings (AT32IDE)	10
Figure 4. AT32F413RCT7 ROM settings (IAR)	11
Figure 5. Relationship diagram between ABZ signals	17
Figure 6. Connection settings	18
Figure 7. AS5047P magnetic encoder transmit data	19
Figure 8. Select open loop control mode	19
Figure 9. Open loop control parameters	19
Figure 10. Modify “ui_wave_param.user_define_a” in mc_isr.c file	20
Figure 11. Diagram parameter setting (confirm magnetic encoder direction)	20
Figure 12. Magnetic encoder direction	21
Figure 13. Magnetic encoder zero-position calibration process	22
Figure 14. Select voltage control mode	22
Figure 15. Voltage control parameters	22
Figure 16. Macro definition of motor nominal current	23
Figure 17. Auto identification of winding parameters	23
Figure 18. Macro definitions of winding parameters	24
Figure 19. Motor rated voltage	24
Figure 20. Current PI controller parameter auto tuning	24
Figure 21. Macro definition of current PI controller parameters	25
Figure 22. Schematic diagram of step current	25
Figure 23. Select ID tune	25
Figure 24. D-axis current PID and step current parameters	26
Figure 25. Diagram parameter settings (ID tune)	26
Figure 26. ID tune waveform	27
Figure 27. D-axis current PI control parameter macro definition	27
Figure 28. Select IQ tune	27
Figure 29. Q-axis current PID and step current parameters	28
Figure 30. Diagram parameter settings (IQ tune)	28
Figure 31. IQ tune waveform	29
Figure 32. Q-axis current PI control parameter macro definition	29
Figure 33. Set target current	29

Figure 34. Parameter settings (current loop debugging).....	30
Figure 35. Current loop debug waveform	30
Figure 36. PID parameters, acceleration and deceleration.....	31
Figure 37. Set target speed	31
Figure 38. Diagram parameter settings (speed control).....	31
Figure 39. Waveform in speed control mode.....	32
Figure 40. Position reference and position measured.....	32
Figure 41. PID settings	33
Figure 42. Set position reference	33
Figure 43. Diagram parameter setting (position loop).....	33
Figure 44. Waveform in position loop control mode	34
Figure 45. FWC PID gains and the maximum negative d-axis current limit	35
Figure 46. Target speed setting (FWC)	35
Figure 47. Diagram parameter settings (FWC)	36
Figure 48. Waveforms in speed control mode (FWC)	36
Figure 49. Example table for Iq_table and Id_table	37
Figure 50. Macro definitions for MTPA/MTPV table generation.....	38
Figure 51. Enable the macro definition IPM_MTPA_MTPV_TABLE	38
Figure 52. Set the per-unit value for the Id current command	38
Figure 53. Diagram parameter settings (MTPA/MTPV)	39
Figure 54. MTPA/MTPV table indices.....	39
Figure 55. MTPA/MTPV table row and column count	40
Figure 56. External voltage control – potentiometer	41
Figure 57. Control source definition (software control (upper)/external voltage control (lower)).....	41
Figure 58. Speed control mode (external voltage control)	42
Figure 59. Torque control mode (external voltage control).....	42
Figure 60. Set target speed in software control mode	43
Figure 61. Select open loop control.....	43
Figure 62. Magnetic encoder calibration parameters.....	44

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)

Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position estimation:

- Magnetic (absolute) encoder

Sensor-based FOC sinusoidal control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Positioning control

Auto tuning functions:

- Motor winding parameters identification
- Current PI controller parameter self-tuning
- Magnetic encoder deviation angle self-calibration

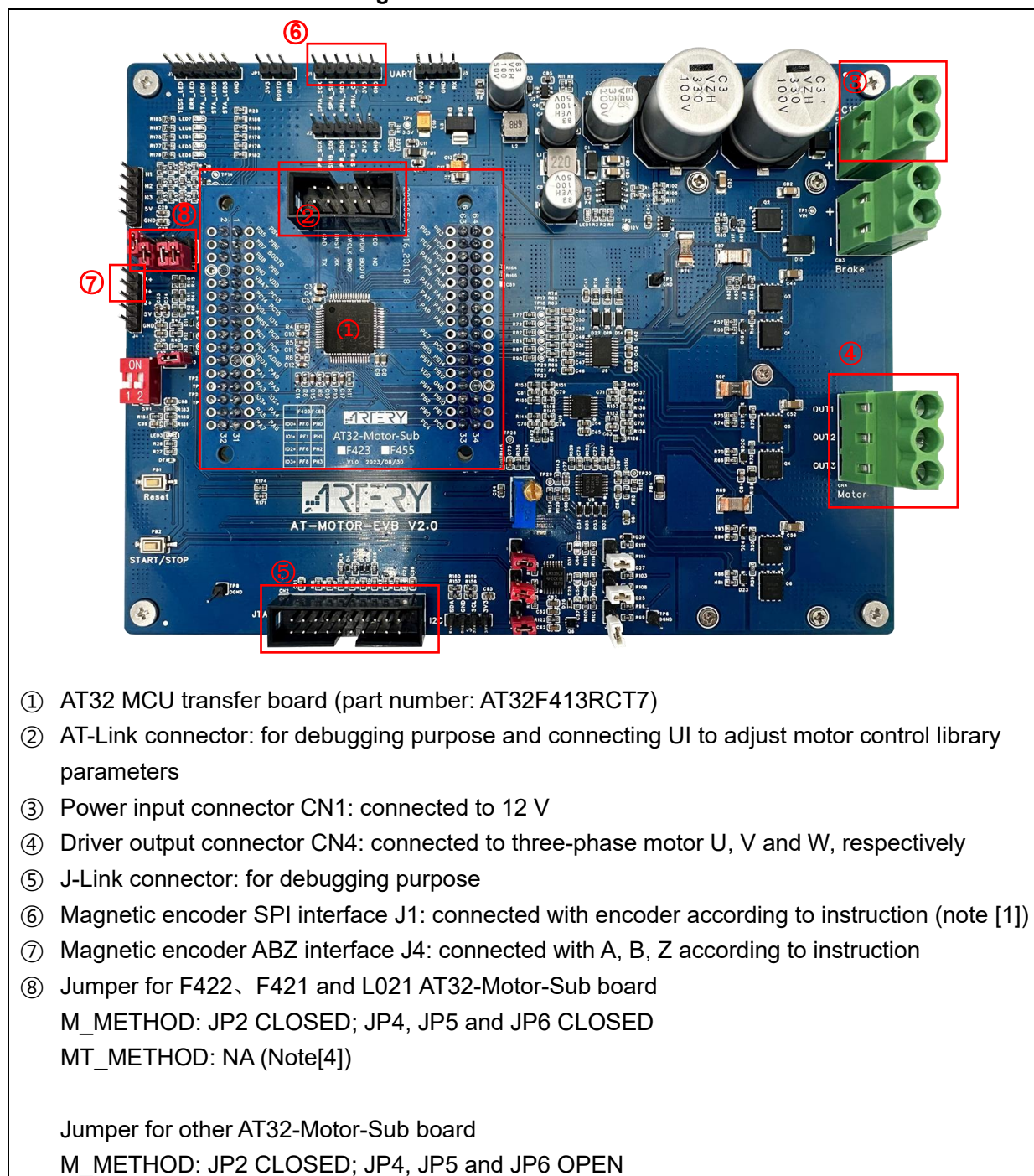
2 Environment requirements

2.1 Hardware requirements

Hardware resources required include a PMSM(BLDC) motor, AT-Link or third-party debugger, and motor control evaluation board (AT-MOTOR-EVB). For details about hardware configurations, please refer to *UM0014_AT32_LV_Motor_Control_EVB_User_Manual_V2.x*.

- PMSM (BLDC) motor
- Magnetic encoder with SPI interface (with/without ABZ interface)
- Debugger
- Motor control EVB: AT-MOTOR-EVB V2.x

Figure 1. AT-MOTOR-EVB V2.x



MT_METHOD: JP2, JP4 and JP5 CLOSED; JP6 OPEN

Note [1]: When SPIA connector is used, remove C16, C17 and C18 capacitors from AT32-MOTOR-Sub and remove R119, R120, R121 and R25 resistors from AT-MOTOR-EVB V2.x board or remove JP13, JP14 and JP15 jumpers from AT-MOTOR-EVB V2.1 board.

2.2 Software requirements

- 1) Take the AT32F413 as an example. Open the demo project under AT32F413_MC_Library_Project\motor_evb_2v0 \at32f413\ pmsm_foc_magnetic_encoder (note [2]).
- 2) Run ArteryMotorMonitor.exe directly (installation not required).
- 3) In Keil IDE, go to **Options - Read/Only Memory Areas** to set ROM according to the Flash memory size of each AT32 MCU, as shown in Table 1 (note [3]). For example, for AT32F413RCT7 with a 256 KB Flash memory, its IROM1 start address is 0x8000000 with a size of 0x3F000, whereas its IROM2 start address is 0x803F000 with a size of 0x1000 (see Figure 2). In AT32IDE, modify *Id file according to the Flash memory size of each AT32 MCU (see Figure 3). In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 4).

Table 1. Correspondence between Flash size and ROM

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EE000	0xFF000	0x7F000	0x6E000	0x3F000
IROM2(address)	0x83EE000	0x80FF000	0x807F000	0x0806E000	0x803F000
IROM2(size)	0x2000	0x1000	0x1000	0x2000	0x1000

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1F400	0x0F400	0x07400	0x03400
IROM2(address)	0x801F400	0x800F400	0x8007400	0x8003400
IROM2(size)	0xC00	0xC00	0xC00	0xC00

Note [2]: This demo project does not support Keil compiler version 5 and O0 optimization level for compiling simultaneously. Please use Keil compiler version 6 or O1 optimization level for compiling. In case of Keil_v5.33, since AT32 BSP source code does not support V6.15 compiler, the Keil compiler version 5 and O1 optimization level should be used.

Note [3]: The magnetic encoder demo project reserves a Flash memory for magnetic encoder deviation angle calibration lookup table; therefore, the correspondence between Flash size and ROM is different from that of other demo projects.

Figure 2. AT32F413RCT7 ROM settings (Keil)

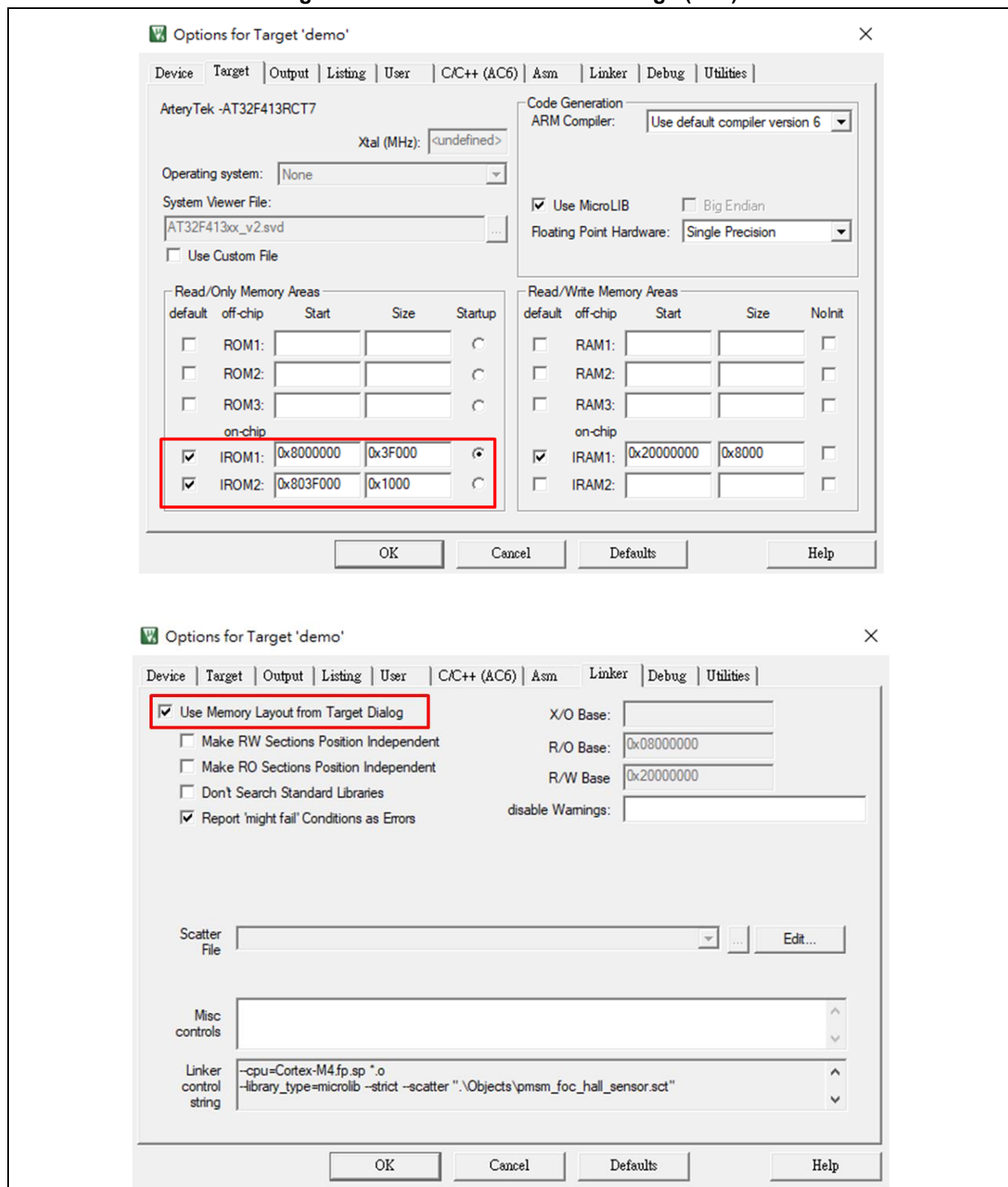


Figure 3. AT32F413RCT7 ROM settings (AT32IDE)

```

25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F000
35     MC_DATA (r)     : ORIGIN = 0x0803F000, LENGTH = 0x1000
36     RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         MC_EncoderStoreAddr = .;
55         *(.MC_EncoderStoreAddr);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 0x800;
58         _MC_VectStoreAddr1 = .;
59         *(.MC_VectStoreAddr1);
60         . = ALIGN(4);
61         . = ORIGIN(MC_DATA) + 0x800 + 800;
62         _MC_VectStoreAddr2 = .;
63         *(.MC_VectStoreAddr2);
64         . = ALIGN(4);
65     } >MC_DATA
66
67     /* The program code and other data goes into FLASH */
68     .text :
69     {
70         . = ALIGN(4);
71         *(.text) /* .text sections (code) */
72         *(.text*) /* .text* sections (code) */
73         *(.glue_7) /* glue arm to thumb code */
74         *(.glue_7t) /* glue thumb to arm code */
75         *(.eh_frame)

```

Figure 4. AT32F413RCT7 ROM settings (IAR)

```

1  /****ICF**** Section handled by ICF editor, don't touch! ****/
2  /*-Editor annotation file-*/
3  /* IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
4  /*-Specials-*/
5  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
6  /*-Memory Regions-*/
7  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
8  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F000-1;
9  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F000;
10 define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 0x800-1);
11 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
12 define symbol __ICFEDIT_region_ROM2_end__ = (__ICFEDIT_region_ROM2_start__ + 800-1);
13 define symbol __ICFEDIT_region_ROM3_start__ = (__ICFEDIT_region_ROM2_end__ + 1);
14 define symbol __ICFEDIT_region_ROM3_end__ = 0x0803F000 + 0x1000-1;
15 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
16 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
17 /*-Sizes-*/
18 define symbol __ICFEDIT_size_cstack__ = 0x400;
19 define symbol __ICFEDIT_size_heap__ = 0x200;
20 /****End of ICF editor section.****ICF****/
21
22 define memory mem with size = 4G;
23 define region ROM region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
24 define region ROM1 region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
25 define region ROM2 region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
26 define region ROM3 region = mem:[from __ICFEDIT_region_ROM3_start__ to __ICFEDIT_region_ROM3_end__];
27 define region RAM region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
28
29 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ {};
30 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ {};
31
32 initialize by copy {readwrite};
33 do not initialize {section .noinit};
34
35 place at address mem: __ICFEDIT_intvec_start__ {readonly section .intvec};
36
37 place in ROM region {readonly};
38 place in ROM1 region {readonly section .MC_EncoderStoreAddr};
39 place in ROM2 region {readonly section .MC_VectStoreAddr1};
40 place in ROM3 region {readonly section .MC_VectStoreAddr2};
41 place in RAM region {readwrite,
42 ..... block CSTACK, block HEAP};

```

3 Quick start guide

Follow the procedures below to fine-tune a PMSM motor with magnetic encoder by using FOC control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Confirm magnetic encoder SPI interface data (see Section 3.3).

Step 4: Select open loop control mode to check whether motor is running normally and to verify the motor's rotation direction and the angular direction of the encoder (see Section 3.4).

Step 5: Zero-position calibration of the magnetic encoder is required upon initial motor debugging or after control board re-programming (see Section 3.5).

Step 6: Select voltage control mode to check whether the motor spins based on encoder position (see Section 3.6).

Step 7: Current control

STEP 1 – Motor winding parameter identification (see Section 3.7)

STEP 2 – Current PI controller parameter auto identification (see Section 3.7)

STEP 3 – ID tune: fine-tuning Kp and Ki values of current loop (see Section 3.8)

STEP 4 – IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.9)

STEP 5 – Current loop control (see Section 3.10)

Step 8: Tune speed control parameters (see Section 3.11).

Step 9: Tune positioning control parameters if necessary (see Section 3.12).

Step 10: Adjust field weakening control parameters as needed (see Section 3.13).

Step 11: (Skip this step if the motor type is a Surface-mounted Permanent Magnet Synchronous Motor (SPMSM)) If the motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM) and requires Maximum Torque per Ampere (MTPA) control and Maximum Torque per Voltage (MTPV) control, it is necessary to perform MTPA/MTPV table creation and lookup table control (see Section 3.14).

Step 12: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.15)

Step 13: Perform magnetic encoder deviation angle self-calibration if needed (see Section 3.16).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* header file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configurations, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board (V1 or V2, optional).

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.x (include <i>mc_hwio_v1.h</i> file)

3. Enable low-side MOS inverted output according to the model of gate driver on hardware. For example, U3315 on AT32_LV_MOTOR_CONTROL_EVB supports inverted output; therefore, the low-side MOS inverted output is enabled by default.

Table 4. Macro definition setting (low-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default; comment to disable)

4. This example is a FOC vector control demonstration project, so there is no need to make changes. This option, however, is reserved for use in code judgment.

Table 5. Macro definition setting (vector control mode)

Definition	Description
FOC_CONTROL	FOC vector control mode

5. Select a current sampling method from below according to user hardware circuit and application requirements.

Table 6. Macro definition setting (current sampling method)

Definition	Description
THREE_SHUNT	3-shunt current sampling
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling

6. If there is a need to use 2-shunt current sampling, select one from below according to user hardware circuit and application requirements; if not, this step is not needed.

Table 7. Macro definition setting (two-shunt current sampling method)

Definition	Description
U_V_SHUNT	U & V shunt
V_W_SHUNT	V & W shunt
U_W_SHUNT	U & W shunt

7. Based on whether the user's hardware circuit uses the low-side MOSFET's $R_{DS(on)}$ for current sampling.

Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)

Definition	Description
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$ (disabled by default)

8. Based on whether the user's requirements call for using dual ADCs for current sampling (Applicable to MCUs with dual or more ADCs; not suitable for ONE_SHUNT configurations).

Table 9. Macro definition setting (dual ADCs current sampling)

Definition	Description
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (disabled by default)

9. Select magnetic encoder with or without ABZ signals (with ABZ signals by default).

Table 10. Macro definition setting (magnetic encoder)

Definition	Description
MAGNET_ENCODER_W_ABZ	Magnetic encoder with ABZ signals
MAGNET_ENCODER_WO_ABZ	Magnetic encoder without ABZ signals

10. Enable or disable reverse encoder counting mode depending on user circuit configuration. This step is not needed for magnetic encoder without ABZ signals (see Section 3.4).

Table 11. Macro definition setting (encoder reverse counting)

Definition	Description
REVERSE_ENCODER_COUNT	Encoder reverse counting (disabled by default)

11. (Skip this step if the magnetic encoder does not output ABZ signals) Select the desired encoder speed measurement method. In most case, the M_METHOD is selected as this method only needs to capture the number of pulse of the encoder. For more accurate speed measurement, the MT_METHOD is an option, which needs an additional timer to obtain the pulse interval time of the encoder in addition to the pulse number (note [4]).

Table 12. Macro definition setting (speed measurement method)

Definition	Description
M_METHOD	M method for speed measurement
MT_METHOD	M/T method for speed measurement

Note [4]: In the AT-MOTOR-EVB board, it uses JP4 and JP5 CLOSED to connect with another timer to configure

MT_METHOD.

12. Enable or disable magnetic encoder deviation angle calibration. It is enabled by default.

Table 13. Macro definition setting (magnetic encoder deviation angle calibration)

Definition	Description
ANGLE_CALIBRATION	Magnetic encoder offset angle calibration (enabled by default)

13. Enable d/q-axis current signal low-pass filter according to user needs.

Table 14. Macro definition setting (d/q-axis current low-pass filter)

Definition	Description
CURRENT_LP_FILTER	d/q-axis current low-pass filter (it is default disabled for 3-shunt/2-shunt current sampling; it is default enabled for 1-shunt current sampling)

14. Enable a/b/c phase current signal low-pass filter according to user needs.

Table 15. Macro definition setting (a/b/c phase current low-pass filter)

Definition	Description
AC_CURRENT_LP_FILTER	a/b/c phase current low-pass filter (disabled by default)

15. Based on the user's motor type, select either a Surface-Mounted Permanent Magnet Synchronous Motor (SPMSM) or an Interior Permanent Magnet Synchronous Motor (IPMSM) (choose one to activate).

Table 16. Macro definition setting (motor type)

Definition	Description
SPMSM	Surface-Mounted Permanent Magnet Synchronous Motor
IPMSM	Interior Permanent Magnet Synchronous Motor

16. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), enable the macro definition IPM_MTPA_MTPV_TABLE to generate MTPA/MTPV lookup table.

Table 17. Macro definition setting (MTPA/MTPV lookup table generation)

Definition	Description
IPM_MTPA_MTPV_TABLE	Generate MTPA/MTPV lookup table (disabled by default)

17. If the user's motor type is an Interior Permanent Magnet Synchronous Motor (IPMSM), generate MTPA/MTPV lookup table and store them in the Id_MTPA_LUT and Iq_MTPV_LUT matrices within the *MTPA_MTPV_table.c* file. Enable the macro definition IPM_MTPA_MTPV_CTRL to execute lookup table-based torque control.

Table 18. Macro definition setting (MTPA/MTPV lookup table-based torque control)

Definition	Description
------------	-------------

IPM_MTPA_MTPV_CTRL	MTPA/MTPV lookup table-based torque control (enabled by default)
--------------------	--

18. If the user's motor type is a Surface-Mounted Permanent Magnet Synchronous Motor (SPMSM), the flux-weakening function can be optionally enabled during speed control based on user requirements.

Table 19. Macro definition setting (field weakening control)

Definition	Description
FIELD_WEAKENING	Filed weakening control (disabled by default)

19. Enable speed limit feature during torque (current) control according to user needs.

Table 20. Macro definition setting (speed limit during torque control)

Definition	Description
TORQUE_CTRL_WITH_SPEED_LIMIT	Speed limit during torque (current) control (disabled by default)

20. Enable current decouple feature during torque (current) control according to user needs.

Table 21. Macro definition setting (current decouple control)

Definition	Description
CURRENT_DECOUPLE_CTRL	Current decouple control (N/A for AT32L021 series) (disabled by default)

21. Enable winding parameter identification. Once enabled, it is used to detect RS and LS of motor windings, and calculate current loop PI control parameters (see Section 3.7).

Table 22. Macro definition setting (motor winding parameter identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameter identification (enabled by default; comment to disable)

22. Enable or disable the Artery motor UI tool according to user needs.

Table 23. Macro definition setting (Artery motor UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor UI tool (enabled by default; comment to disable)

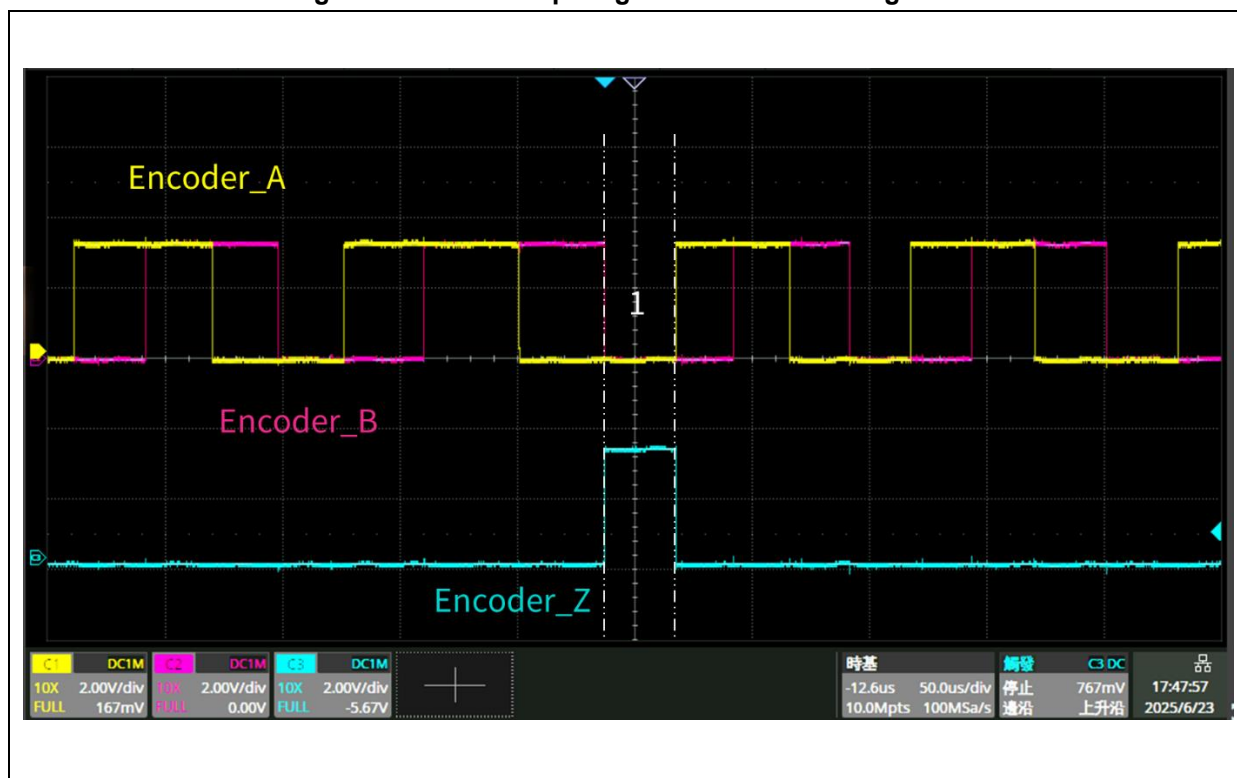
- Table 24 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 24. Motor parameter definition

Definition	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value (unit: V/rpm) (fill in this value when current decouple control is enabled)
LD_LQ_RATIO	Ld to Lq ratio
NOMINAL_CURRENT	Nominal current (unit: ampere)
ABS_ENCODER_RES	Magnetic encoder resolution (unit: bits; two to the power of bit)
ABS_ENCODER_IIF_RES	Magnetic encoder incremental interface resolution (unit: count per revolution)
ABS_ENCODER_SAMPLE_TIME	Magnetic encoder SPI interface data transmit time (unit: us)
ENC_IDX_COUNT	Encoder index signal width (unit: count) (note [5])
ENC_STALL_TIME	Encoder stall time (unit: ms)

Note [5]: This applies to ABZ signals with zero index mode of the magnetic encoder. Figure 5 illustrates the relationship between ABZ signals of the AS5047P magnetic encoder. The width between the rising edge and falling edge of the Z signal is 1 count aligned with AB signals, so the ENC_IDX_COUNT is set to 1 (usually 1 or 2 counts).

Figure 5. Relationship diagram between ABZ signals



3.2 Connection settings

After the hardware and software are well prepared, set up connection between UI and the control board as follows (see AN0063 for details).

STEP-1

Connect the motor, AT-Link/J-Link and board power supply to the motor development board, and connect USART interface to PC via an USB cable.

STEP-2

Use MDK to compile demo project code, and use J-Link or AT-Link to download to the on-board chip.

STEP-3

Run ArteryMotorMonitor.exe: Click File -> Open Project and select ArteryMotorMonitor.atmcx->Open.

STEP-4

Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 6.

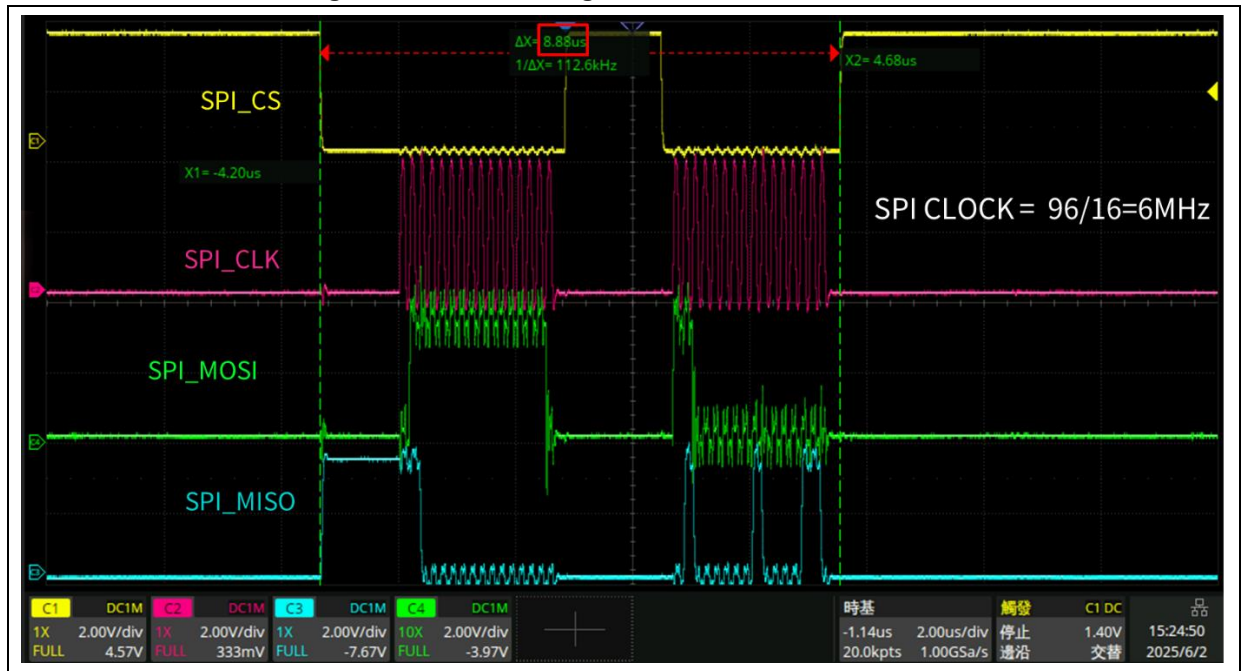
Figure 6. Connection settings



3.3 Magnetic encoder SPI interface

This demo project uses AS5047P magnetic encoder, which can be configured according to the SPI interface macro definition settings (AS5047P.h), relevant function files (AS5047P.c) and the selected magnetic encoder. Obtain the required data and then check if it is correct. For example, the data "0 ~ (magnetic encoder resolution -1)" should be obtained after the motor rotates one revolution. If the magnetic encoder does not output ABZ signals (MAGNET_ENCODER_WO_ABZ), users need to confirm the data transmit time of SPI interface. For example, when SPI CLOCK = 6 MHz, the data transmit time of AS5047P magnetic encoder is 8.88us, as shown in Figure 7. In this case, set the ABS_ENCODER_SAMPLE_TIME value to be larger than 8.88us, 9us for example, to ensure obtaining the latest position data at motor control interrupt.

Figure 7. AS5047P magnetic encoder transmit data

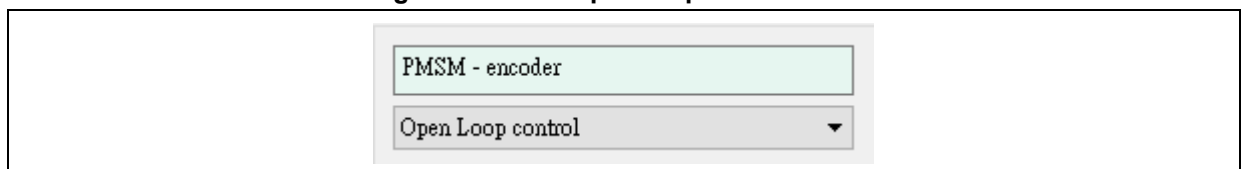


3.4 Open loop control

With open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction, as well as the encoder mode's rotational accuracy. The open-loop voltage and angular increment are adjusted incrementally from zero, based on the motor's speed and phase current.

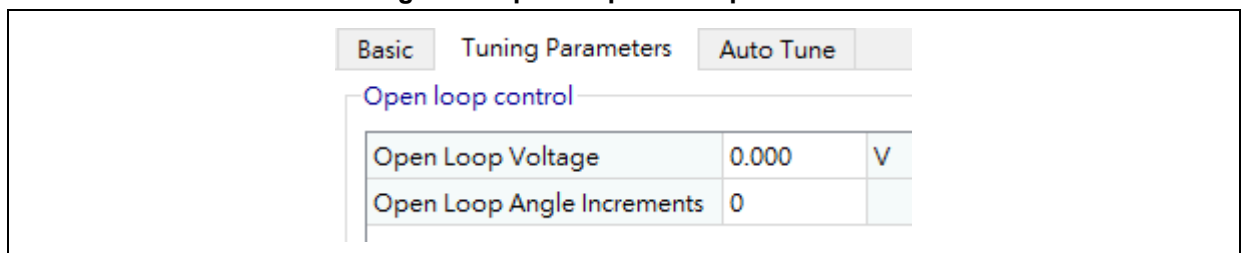
STEP-1: Select "Open Loop control" mode.

Figure 8. Select open loop control mode



STEP-2: Go to "Tuning Parameters" to set "Open Loop Voltage" and "Open Loop Angle Increments". Click on "Start Motor" and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

Figure 9. Open loop control parameters



Tips:

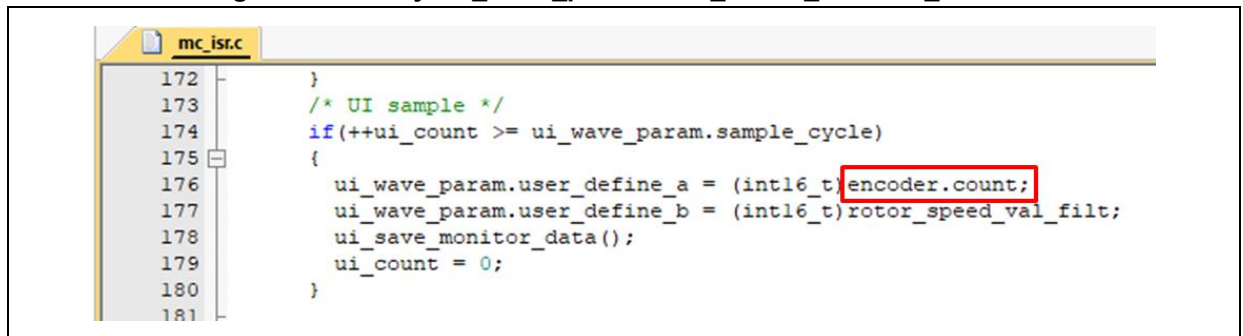
Initially, set “Open Loop Voltage” value to 0.5~2V and set “Open Loop Angle Increments” to 1~10, and monitor whether the motor starts to run. If not, check whether the 3-phase cables are correctly connected.

STEP-3: After the motor starts to run properly, check if it is running in the right direction. If it is reversed, the user needs to change the two phases in the three phases.

STEP-4: Confirm whether the magnetic encoder direction is correct

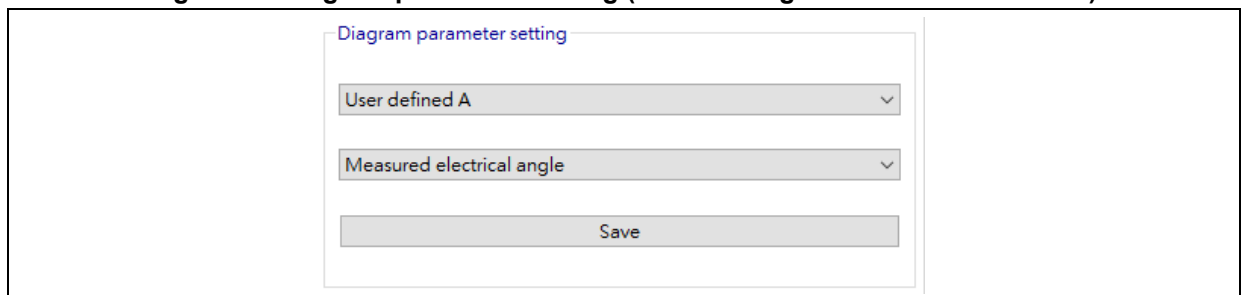
STEP-4-1: Modify “ui_wave_param.user_define_a = (int16_t)encoder.count;” in the *mc_isr.c* file, and then recompile and re-program the code.

Figure 10. Modify “ui_wave_param.user_define_a” in mc_isr.c file



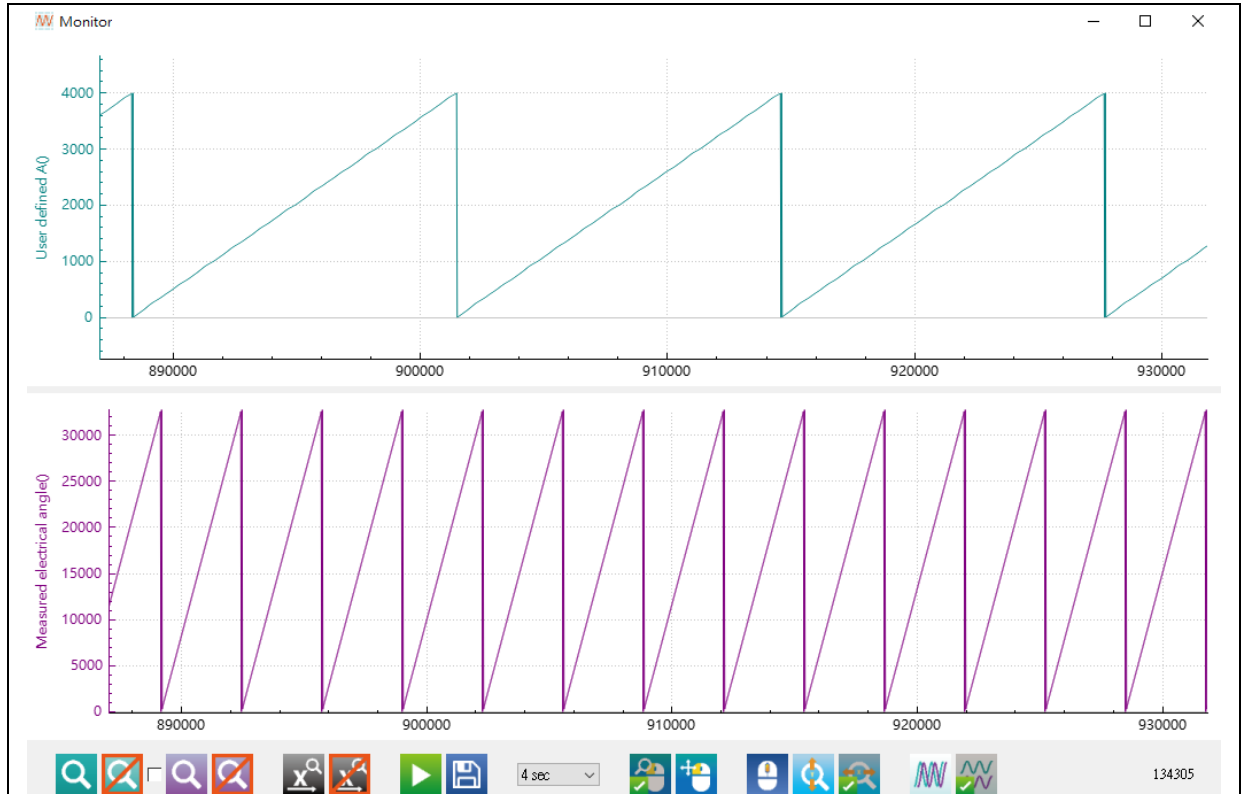
STEP-4-2: Set “User defined A” and “Measured electrical angle”, and click on “Save”.

Figure 11. Diagram parameter setting (confirm magnetic encoder direction)



STEP-4-3: After the motor starts to run in open loop control mode, check whether the “User defined A” is the shape of right triangle as shown in Figure 12. If not, it means that the magnetic encoder direction is reversed. For magnetic encoder with ABZ signals (MAGNET_ENCODER_W_ABZ), the direction can be changed by setting REVERSE_ENCODER_COUNT (Table 11) in *motor_control_drive_param.h* file. If the magnetic encoder does not include ABZ signals (MAGNET_ENCODER_WO_ABZ), swapping any two phases of the motor's three-phase wiring will reverse the motor's rotation direction, thereby altering the magnetic encoder's rotation direction.

Figure 12. Magnetic encoder direction



3.5 Magnetic encoder zero-position calibration

Zero-position calibration of the magnetic encoder is required upon initial motor debugging or after control board re-programming. First, confirm that ABS encoder is selected as the firmware control mode, as shown in Figure 13. Then, select “Voltage Control” and set encoder align voltage. Click on “Encoder Align” to start zero-position calibration. If magnetic encoder without ABZ signals is selected (MAGNET_ENCODER_WO_ABZ), the motor is fixed on the D axis for performing zero-position calibration. If magnetic encoder with ABZ signals is selected (MAGNET_ENCODER_W_ABZ), the motor keeps slowly running until it detects the zero position before stop, that is, the end of the encoder zero-position calibration (note [6]). Users can check the “Encoder offset” to confirm whether it is getting closer to the aligned value. If there is big difference, users can fine-tune the voltage and re-start zero-position calibration (note [7]).

Note [6]: If encoder error occurs, it is necessary to check whether the motor’s encoder cable is being connected with the EVB encoder interface. In case of error, it is possible to remove it by clicking on “Fault Ack”.

Note [7]: Absolute position can be obtained with magnetic encoder, therefore, after zero-position calibration, the code stores the “Encoder offset” value to the FLASH, which allows to start the motor without re-calibration after control board repowering.

Figure 13. Magnetic encoder zero-position calibration process

Status: ESC_STATE_SAFETY_READY

no error

over voltage error

under voltage error

over temperature error

over current error

encoder error

hall error

parameter identify error

hall learn error

startup error

Command

Start Motor

Stop Motor

Encoder Align

Fault Ack

Write Flash

SPMSM - ABS encoder

Voltage Control

Encoder align

Encoder Align Voltage 0.301 V

Encoder Offset 277

Align process state NOT RUN YET

Voltage control

Vq reference 0.000 V

Vd reference 0.000 V

log

Save

Clear

1 13:56:01 Load ok

2 13:56:01 Set REG 8 = 1 ok

3.6 Voltage control

After the encoder alignment is complete, it is possible to perform voltage control based on encoder position.

STEP-1: Select "Voltage Control".

Figure 14. Select voltage control mode

PMSM - encoder

Voltage Control

STEP-2: Use Q-axis voltage control to drive the motor, and check whether the Ia and Ib values match the actual currents, whether the current offset value is closer to 0 and whether the Ia phase leads Ib phase by 120 degrees (see Figure 35).

Figure 15. Voltage control parameters

Voltage control

Vq reference 1 V

Vd reference 0 V

3.7 Auto identification and tuning of parameters

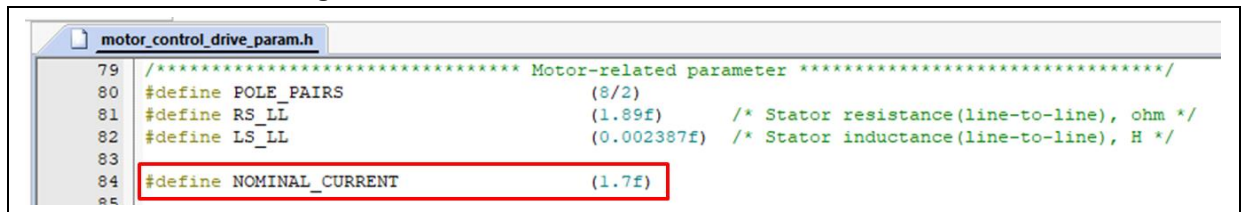
Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters.

Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

Auto identification of motor winding parameters:

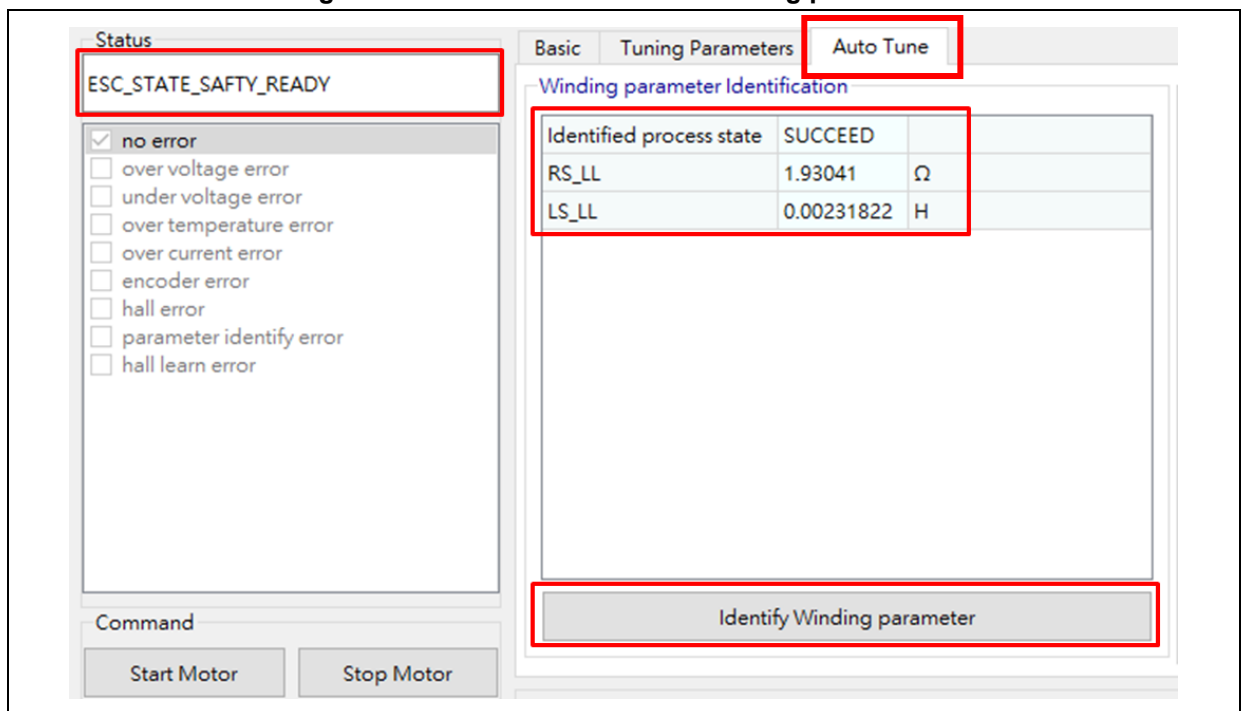
STEP-1: Fill the macro definition NOMINAL_CURRENT into *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 16.

Figure 16. Macro definition of motor nominal current



STEP-2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” in “safety ready” status to perform winding parameter identification, as shown in Figure 17.

Figure 17. Auto identification of winding parameters



STEP-3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 18.

Figure 18. Macro definitions of winding parameters

```

89
90 /***** Motor-related parameter *****/
91 /* Motor parameters */
92 #define RS_LL (1.89) /* Stator resistance, ohm */
93 #define LS_LL (0.002387) /* Stator inductance, H */

```

Current PI controller parameter auto tuning:

STEP-1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 19.

Figure 19. Motor rated voltage

```

100
101 /***** Drive-related parameter *****/
102 /* basic */
103 #define VDC_RATED (24.0f)
104 #define V_SENSE_GAIN (10/(3.9f+180+10)) // 0.05157
105 #define ADC_REFERENCE_VOLT (3.3f)
106 #define ADC_DIGITAL_SCALE_12BITS (4095)

```

STEP-2: Go to Auto Tune window, and click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, current Q axis and D axis PI controller parameters are automatically updated, as shown in Figure 20.

Figure 20. Current PI controller parameter auto tuning

Auto-tune PI parameter of Current controller

Torque KP	7717
Torque KI	6110
Torque KP DIV	512
Torque KI DIV	8192

Auto-tune Current PI parameter

STEP-3: Fill the parameters shown in Figure 20 into the corresponding macro definition (*motor_control_drive_param.h*), then re-compile and re-program code, as shown in Figure 21.

Figure 21. Macro definition of current PI controller parameters

```

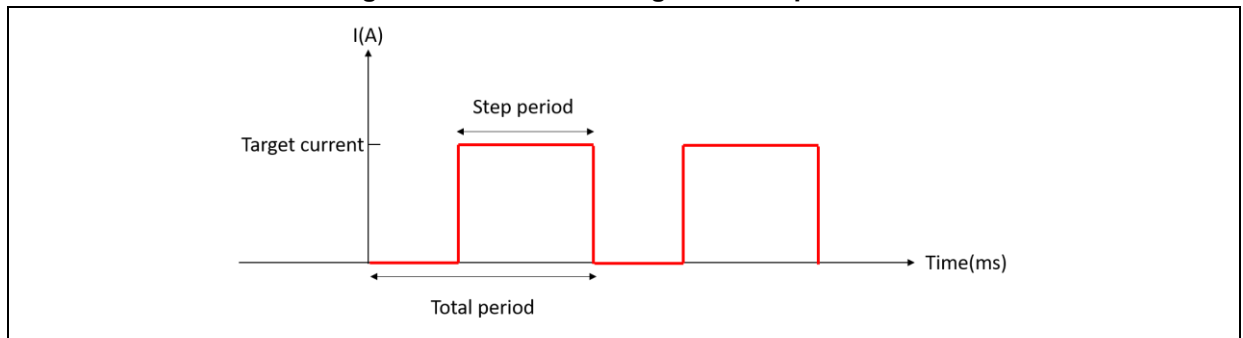
motor_control_drive_param.h
244  /* Id/Iq pid parameter */
245  #define PID_ID_KP_DEFAULT          (7717)
246  #define PID_ID_KI_DEFAULT          (6110)
247  #define PID_ID_KP_GAIN_DIV         (512)
248  #define PID_ID_KP_GAIN_DIV_LOG     (LOG2(PID_ID_KP_GAIN_DIV))
249  #define PID_ID_KI_GAIN_DIV         (8192)
250  #define PID_ID_KI_GAIN_DIV_LOG     (LOG2(PID_ID_KI_GAIN_DIV))
251
252  #define PID_IQ_KP_DEFAULT          (7717)
253  #define PID_IQ_KI_DEFAULT          (6110)
254  #define PID_IQ_KP_GAIN_DIV         (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG     (LOG2(PID_IQ_KP_GAIN_DIV))
256  #define PID_IQ_KI_GAIN_DIV         (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG     (LOG2(PID_IQ_KI_GAIN_DIV))

```

3.8 ID tune

A step current is generated in ID Tune mode, as shown in Figure 22. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of D axis current.

Figure 22. Schematic diagram of step current



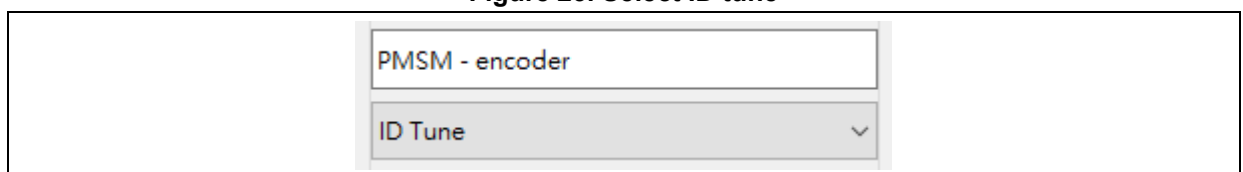
Note: It is recommended to tune D-axis current and then Q-axis current, and to set KP and KI to 0 for D-axis and Q-axis current to avoid unmatched default values affecting current parameter adjustment.

Note: It is recommended to set D-axis current command to 5-10% of the rated current and then slowly increase the D-axis current to the target value to align the rotor with D-axis, so that to protect IPM or MOSFET components from being burnt out by current surges.

Follow the steps bellow:

STEP-1: Select "ID tune".

Figure 23. Select ID tune



STEP-2: Go to “Tuning Parameters: window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 24.

Figure 24. D-axis current PID and step current parameters

Basic	Tuning Parameters	Auto Tune
Unit step config		
Current Tune target current	0.999	A
Current Tune total period	100	ms
Current Tune step period	2	ms
ID Current control		
Flux KP	7500	
Flux KI	6000	
Flux KP DIV	512	
Flux KI DIV	8192	

STEP-3: Click on “Start Motor”.

STEP-4: Set “Flux reference(Id)” and “Flux measured(Id)” in “Diagram parameter setting”, and then click on “Save”.

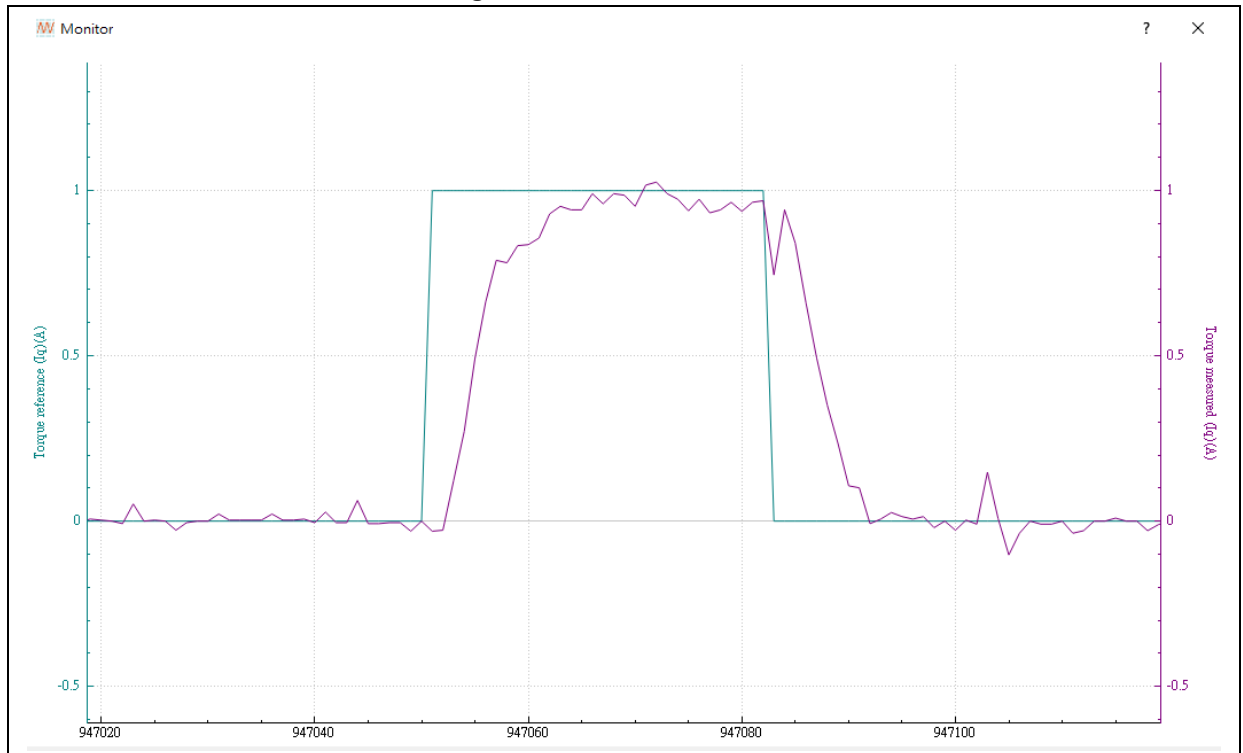
Figure 25. Diagram parameter settings (ID tune)

Diagram parameter setting
Flux reference (Id) ▼
Flux measured (Id) ▼
Save

STEP-5: Click on  icon to generate waveforms.

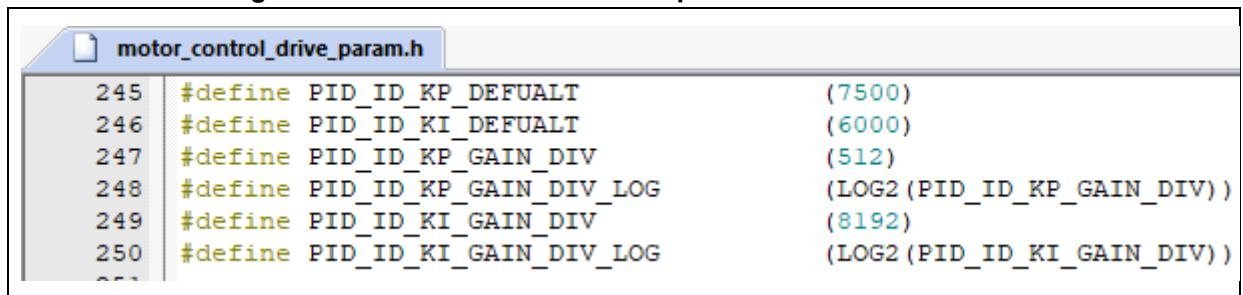
STEP-6: Check whether the current response is as expected, as shown in Figure 31. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 26. ID tune waveform



STEP-7: After ID tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 27.

Figure 27. D-axis current PI control parameter macro definition



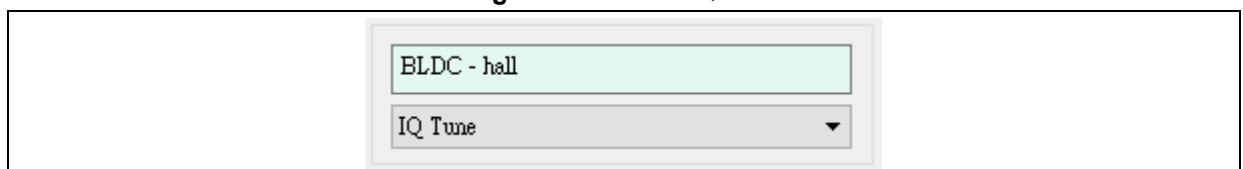
3.9 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 22. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

Follow the steps below:

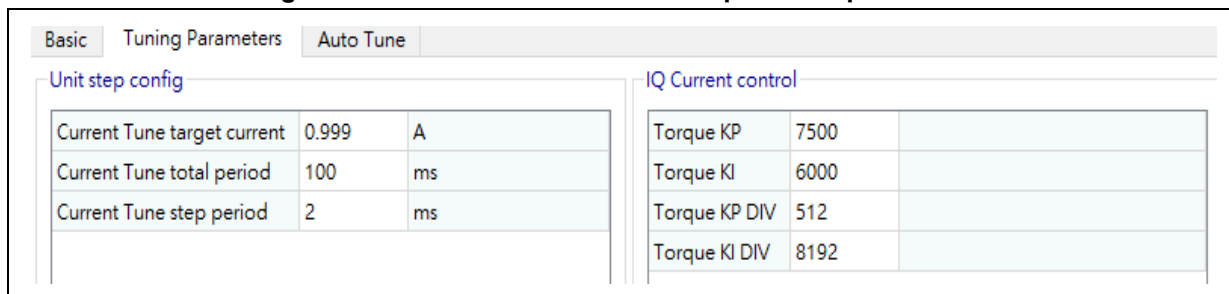
STEP-1: Select "IQ tune".

Figure 28. Select IQ tune



STEP-2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 29.

Figure 29. Q-axis current PID and step current parameters



Unit step config			IQ Current control	
Current Tune target current	0.999	A	Torque KP	7500
Current Tune total period	100	ms	Torque KI	6000
Current Tune step period	2	ms	Torque KP DIV	512
			Torque KI DIV	8192

STEP-3: Click on “Start Motor”.

STEP-4: Set “Torque reference(Iq)” and “Torque measured(Iq)” in “Diagram parameter setting”, and then click on “Save”.

Figure 30. Diagram parameter settings (IQ tune)

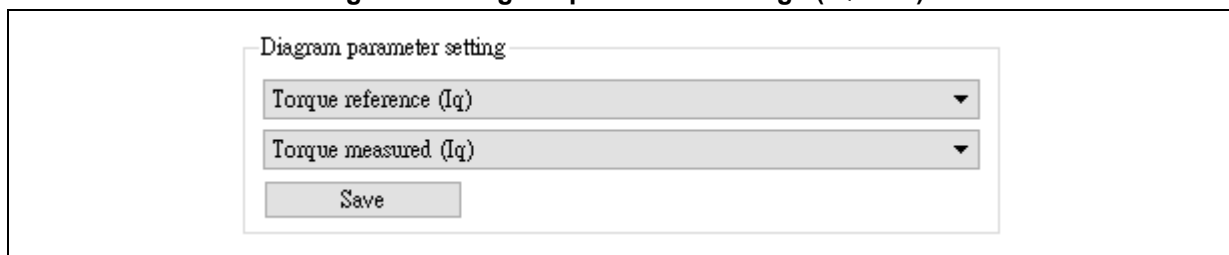


Diagram parameter setting

Torque reference (Iq) ▼

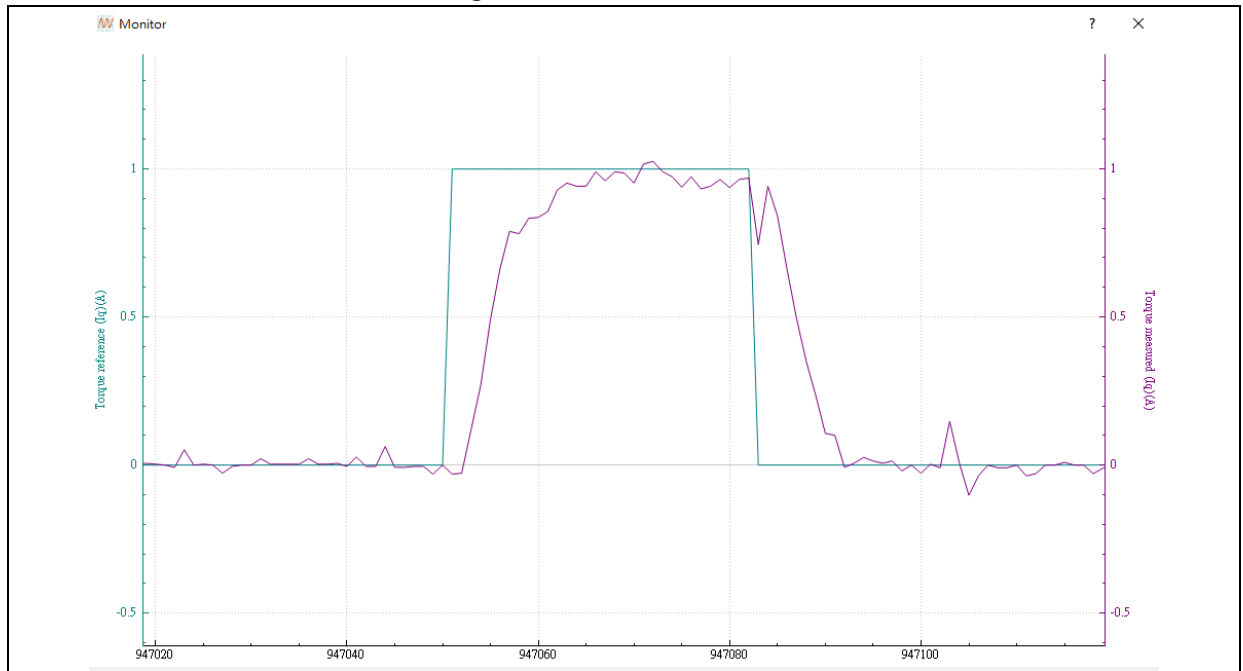
Torque measured (Iq) ▼

Save

STEP-5: Click on  icon to generate waveforms. For more information, please refer to *AN0063 AT32 Motor Monitor Application Note*.

STEP-6: Check whether the current response is as expected, as shown in Figure 31. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 31. IQ tune waveform



STEP-7: After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 32.

Figure 32. Q-axis current PI control parameter macro definition

```

motor_control_drive_param.h
252 #define PID_IQ_KP_DEFAULT      (7500)
253 #define PID_IQ_KI_DEFAULT      (6000)
254 #define PID_IQ_KP_GAIN_DIV     (512)
255 #define PID_IQ_KP_GAIN_DIV_LOG (LOG2 (PID_IQ_KP_GAIN_DIV))
256 #define PID_IQ_KI_GAIN_DIV     (8192)
257 #define PID_IQ_KI_GAIN_DIV_LOG (LOG2 (PID_IQ_KI_GAIN_DIV))

```

3.10 Current loop control

In this mode, the current loop control is used to control torque by tuning the target current. The results are reflected through waveforms. Follow the steps below:

STEP-1: Select "Torque Control".

STEP-2: Set software control as control source and configure the target current reference (Torque reference).

Figure 33. Set target current

Torque reference (Iq)	0.100	A
Flux reference (Id)	0.000	A

STEP-4: Click on "Start Motor".

STEP-5: Set “Ia” and “Ib”, and then click on “Save”.

Figure 34. Parameter settings (current loop debugging)

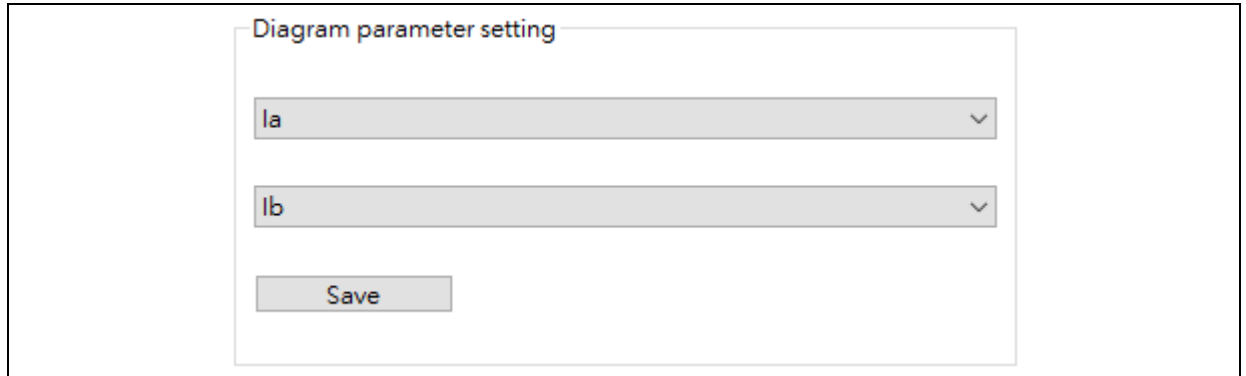


Diagram parameter setting

Ia

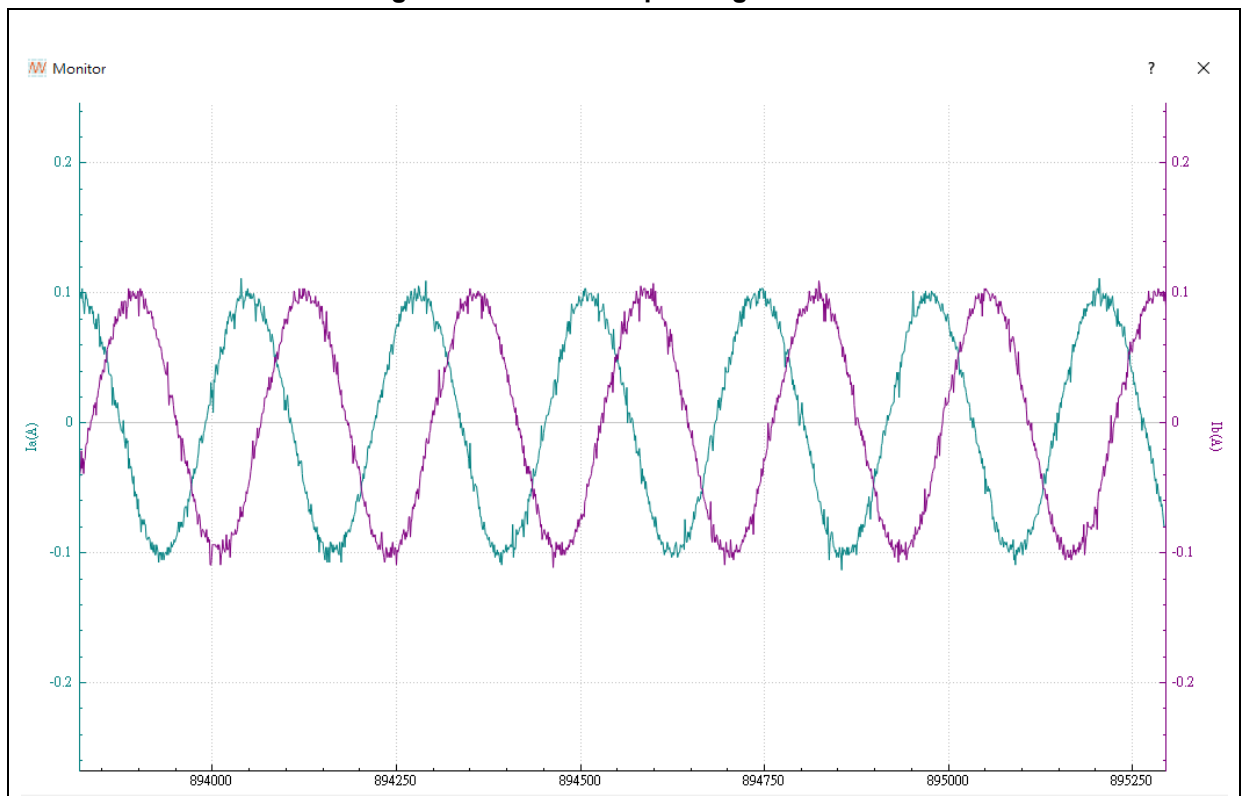
Ib

Save

STEP-6: Click on  icon to open a waveform window.

STEP-7: Check if the current response is as expected, as shown in Figure 35.

Figure 35. Current loop debug waveform



3.11 Speed loop control

In this mode, users can set the speed PID parameters, acceleration and deceleration, and check the response through waveform. Follow the steps below:

STEP-1: Select “Speed Control”.

STEP-2: Go to “Tuning Parameters” window, and set speed PID parameters, acceleration and deceleration.

Figure 36. PID parameters, acceleration and deceleration

Speed control		
Speed KP	1000	
Speed KI	4	
Speed KP DIV	1024	
Speed KI DIV	1024	
Speed acceleration	8	rpm/ms
Speed deceleration	8	rpm/ms

STEP-3: Set software control as control source and configure the target speed reference (Speed reference).

Figure 37. Set target speed

Target speed		
Speed reference	0	rpm

STEP-4: Click on “Start Motor”.

STEP-5: Set “Speed reference” and “Speed measured”.

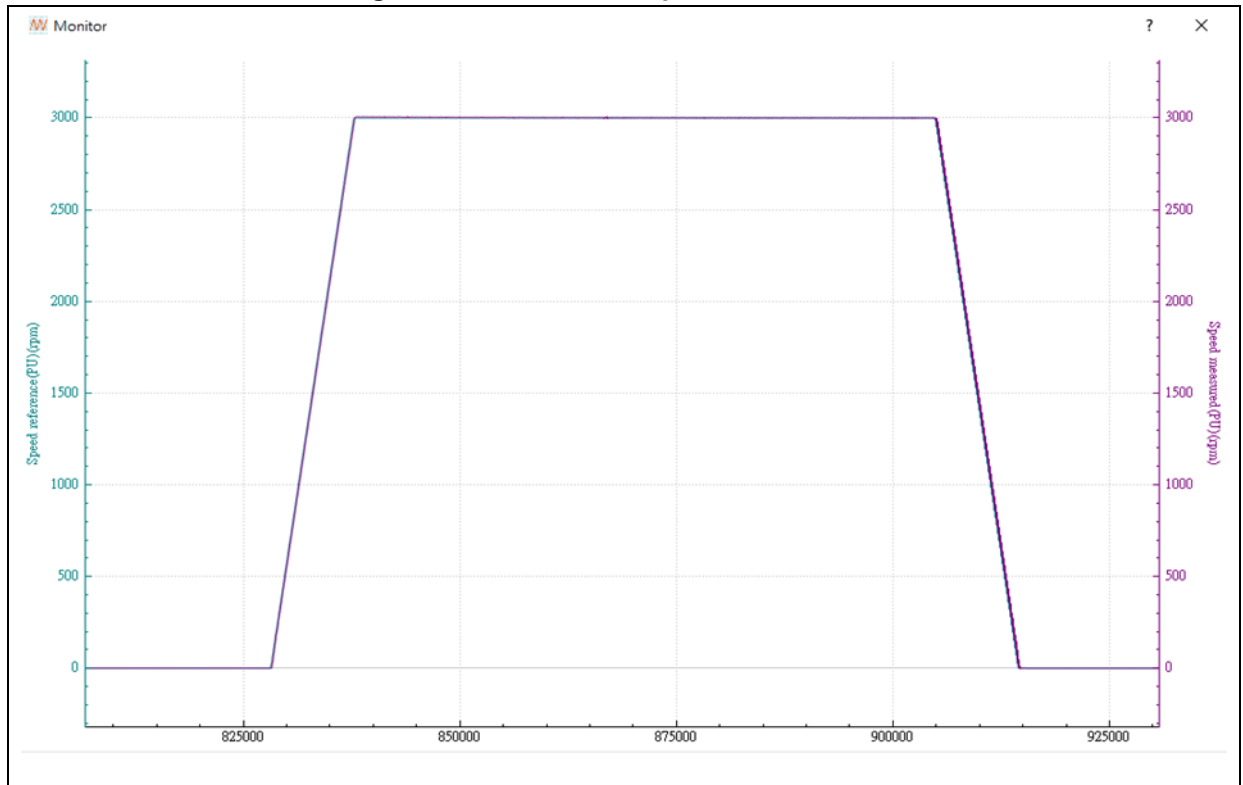
Figure 38. Diagram parameter settings (speed control)

Diagram parameter setting	
Speed reference(PU)	▼
Speed measured(PU)	▼
Save	

STEP-6: Click on  icon to open a waveform window.

STEP-7: Check if the speed response is as expected, as shown in Figure 39. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 39. Waveform in speed control mode



Note: The maximum displayed values for speed waveforms (Speed reference(PU) and Speed measured(PU)) are defined as $1.25 \times \text{MAX_SPEED_RPM}$, with the minimum being $-1.25 \times \text{MAX_SPEED_RPM}$. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to set MAX_SPEED_RPM to the motor's rated speed to avoid display anomalies.

3.12 Position loop control

In position loop control mode, the user can set position loop PID data and view response waveforms. Follow the steps below:

STEP-1: Select "Position Control" through the drop-down menu. If the motor has rotated, its angle usually will not stop exactly at the Zero degree. In this case, the positioning command reference and the measured position are the angle after the motor's rotation.

Figure 40. Position reference and position measured



STEP-2: Set position controller PID parameters.

Figure 41. PID settings

Position controller	
Position KP	80
Position KI	0
Position KI stable	800
Position KD	0
Position KP DIV	32768
Position KI DIV	65536
Position KD DIV	65536

STEP-3: Set position reference, such as 36000 degrees.

Figure 42. Set position reference

Angle		
Position reference	0	degree

STEP-4: Click on “Start Motor”.

STEP-5: Set “Position reference(PU)” and “Position measured(PU)”, and then click on “Save”.

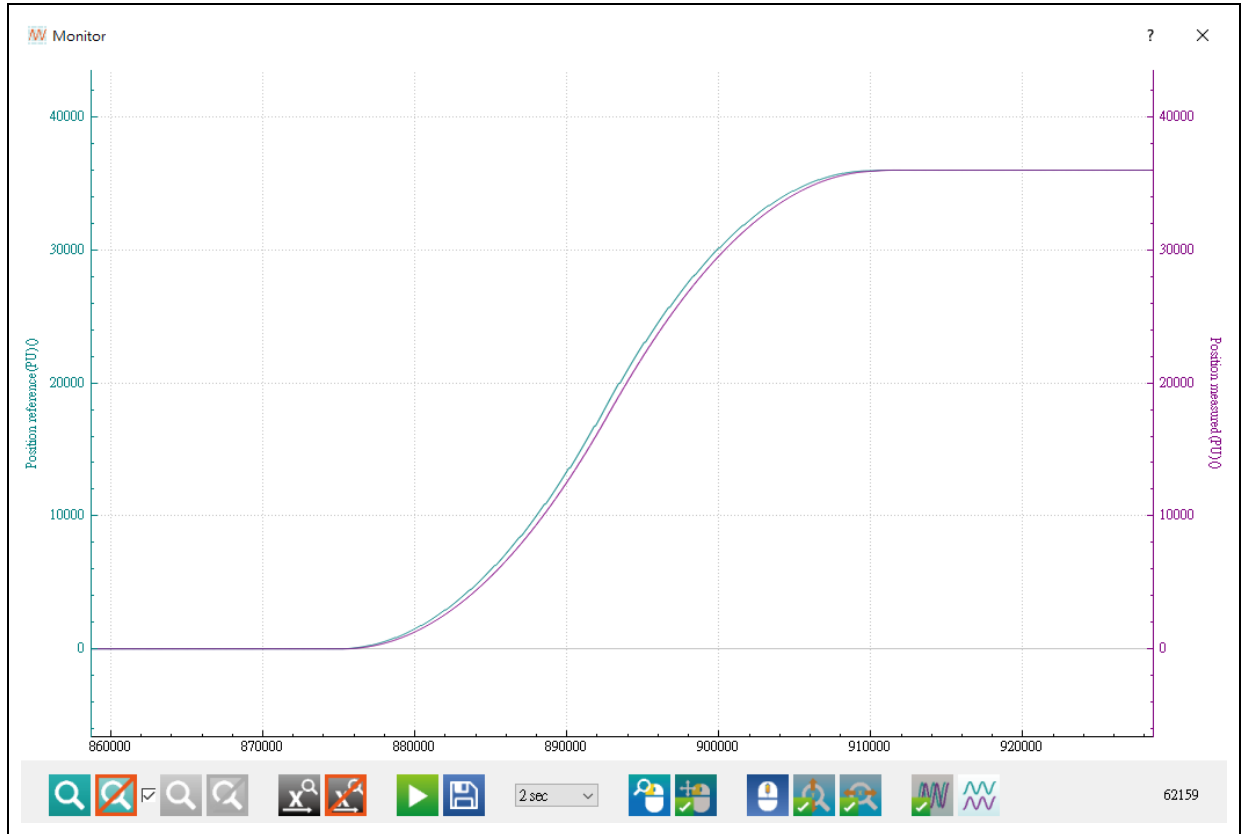
Figure 43. Diagram parameter setting (position loop)

Diagram parameter setting	
Position reference(PU)	▼
Position measured(PU)	▼
Save	

STEP-6: Click on  icon to open a waveform window.

STEP-7: Check whether the position response is as expected, as shown in Figure 44. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 44. Waveform in position loop control mode



Note: The maximum displayed values for position waveforms (Position reference(PU) and Position measured(PU)) are defined as $1.25 \times \text{MAX_POSITION_ANGLE}$, with the minimum being $-1.25 \times \text{MAX_POSITION_ANGLE}$. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to adjust MAX_POSITION_ANGLE value appropriately to avoid display anomalies.

3.13 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM)

This mode supports Surface-Mounted Permanent Magnet Synchronous Motors (SPMSM). During speed-loop control, it injects negative d-axis current commands to weaken the permanent magnet flux linkage, thereby extending the motor's operational speed range. Adjustable parameters include the flux-weakening control (FWC) PID gains and the maximum negative d-axis current limit. After tuning, the dynamic response can be visualized via waveform plotting. Detailed implementation steps are as follows:

STEP-1: Select "Speed Control".

STEP-2: Go to "Tuning Parameters" window to configure the maximum d-axis (flux-weakening) current command and adjust the flux-weakening PID parameters.

Note: Configure the maximum d-axis (flux-weakening) current command according to the motor's rated specifications. Exceeding this limit may cause overheating damage to the motor.

Figure 45. FWC PID gains and the maximum negative d-axis current limit

-Flux weakening tuning		
Max. Flux weakening Current	1.4	A
Flux weakening KP	300	
Flux weakening KI	100	
Flux weakening KP DIV	2048	
Flux weakening KI DIV	32768	

STEP-3: Select software control as the control source and input the speed reference value, such as 10000rpm.

Note: The flux-weakening control (FWC) is triggered only when the motor speed exceeds the base speed (i.e., the maximum sustainable speed under rated flux linkage).

Figure 46. Target speed setting (FWC)

Target speed		
Speed reference	0	rpm

STEP-4: Select "Speed reference (PU)" and "Speed measured (PU)".

Figure 47. Diagram parameter settings (FWC)

Diagram parameter setting

Speed reference(PU)
▼

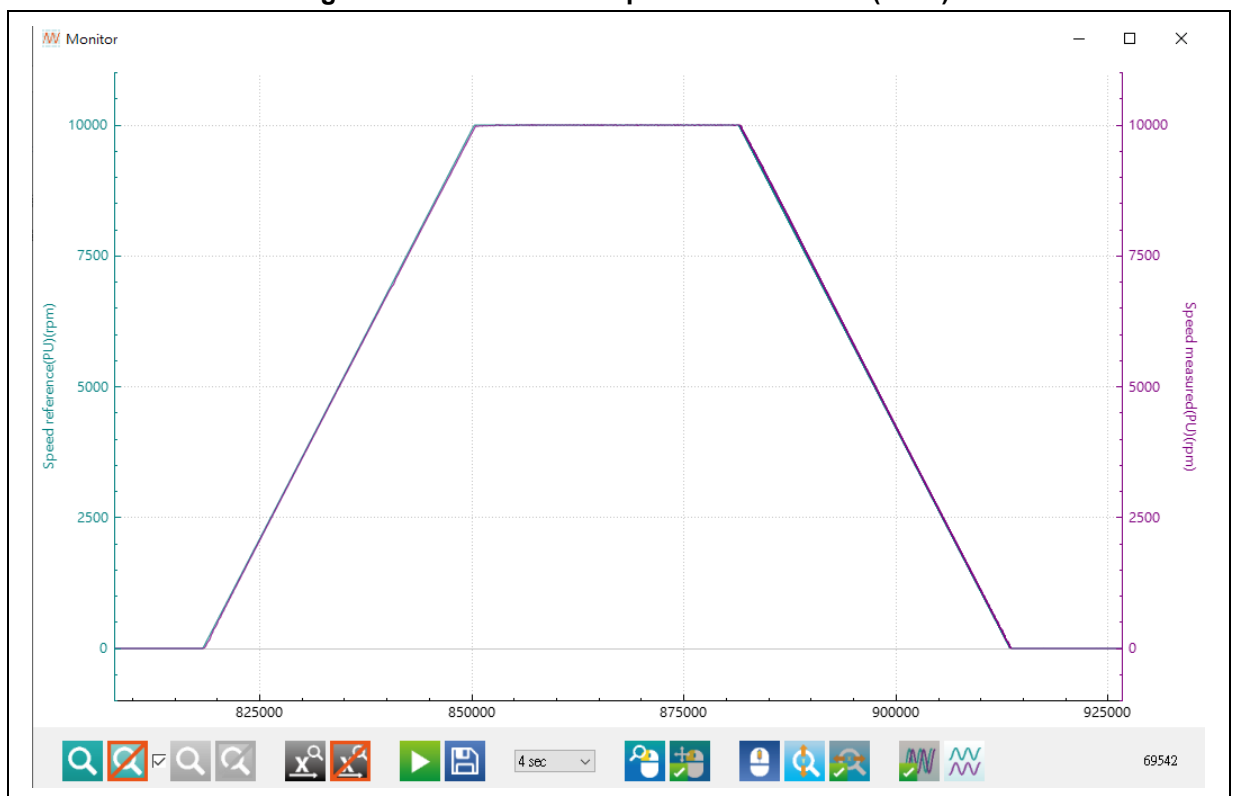
Speed measured(PU)
▼

Save

STEP-5: Click on  icon to open a waveform window.

STEP-6: Check whether the speed response is as expected, as shown in Figure 48. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 48. Waveforms in speed control mode (FWC)



3.14 MTPA/MTPV control (Interior Permanent Magnet Synchronous Motor, IPMSM)

MTPA (Maximum Torque Per Ampere) is an optimized control strategy for Interior Permanent Magnet Synchronous Motors (IPMSM) in the base speed region (constant torque zone). Its goal is to minimize the stator current amplitude for a given torque demand by adjusting the d-axis and q-axis current components (I_d , I_q), ensuring the motor delivers the target torque with minimal current.

MTPV (Maximum Torque Per Voltage) is a deep flux-weakening control strategy for the high-speed region, where the motor maximizes torque output under voltage constraints by adjusting I_d and I_q . At high speeds, the back-EMF increases, reducing voltage margin. This necessitates increasing the negative I_d current to weaken the magnetic field and extend the speed range.

Why MTPA/MTPV is exclusive to IPMSM:

IPMSM relies on salient-pole characteristics ($L_d < L_q$) to generate reluctance torque, which is critical for MTPA/MTPV.

Surface PMSM (SPMSM) cannot leverage this due to structural limitations ($L_d \approx L_q$) making MTPA/MTPV ineffective.

Implementation Steps for MTPA/MTPV Table Generation:

Step 1: Define parameters for table generation: Maximum torque, Maximum speed, Torque step size and Speed step size. If a specific torque/speed value is not explicitly listed in the table, the corresponding d-axis and q-axis current components (I_d , I_q) will be calculated via interpolation.

STEP-2: Generate I_q _table and I_d _table, as shown in Figure 49.

Example parameters for table generation:

Maximum torque: 5 N·m

Torque step size: 0.5 N·m

Maximum speed: 5000 rpm

Speed step size: 500 rpm

Figure 49. Example table for I_q _table and I_d _table

Torque(Nm) Speed(rpm)	0	0.5		5
500	$I_{q_{pu1}}$	$I_{q_{pu2}}$		
1000				
⋮				
5000				

Torque(Nm) Speed(rpm)	0	0.5		5
500	$-I_{d_{pu1}}$	$-I_{d_{pu2}}$		
1000				
⋮				
5000				

Note: The conversion formula between per-unit current values and actual physical current values is:

$$I(A) = \text{current_base} \cdot I_{pu} / 32767$$

STEP-3: Configure Motor Type as SPMSM in *motor_control_param.h* file, disable FIELD_WEAKENING and IPM_MTPA_MTPV_TABLE, as shown in Figure 50.

Figure 50. Macro definitions for MTPA/MTPV table generation

```

95  /* motor type */
96  #define SPMSM
97  //#define IPMSM
98
99
100 #if defined SPMSM
101  //#define FIELD_WEAKENING
102  //#define IPM_MTPA_MTPV_TABLE
103  #elif defined IPMSM
104  #define IPM_MTPA_MTPV_CTRL
105  #endif

```

STEP-4: Mount the motor on the dynamometer platform and verify mechanical alignment to minimize vibration and torque measurement errors. Complete speed-loop tuning to ensure stable motor operation under both no-load and load conditions)

STEP-5: Enable macro definition IPM_MTPA_PTPV_TABLE, and then enter the debug mode.

Figure 51. Enable the macro definition IPM_MTPA_MTPV_TABLE

```

    #if defined SPMSM
    //#define FIELD_WEAKENING
    #define IPM_MTPA_MTPV_TABLE
    #elif defined IPMSM
    #define IPM_MTPA_MTPV_CTRL
    #endif

```

STEP-6: Configure -Id current reference (current.lqdref.d) via watch window, the Iq reference (current.lqdref.q) is automatically generated by the speed PI controller.

Figure 52. Set the per-unit value for the Id current command

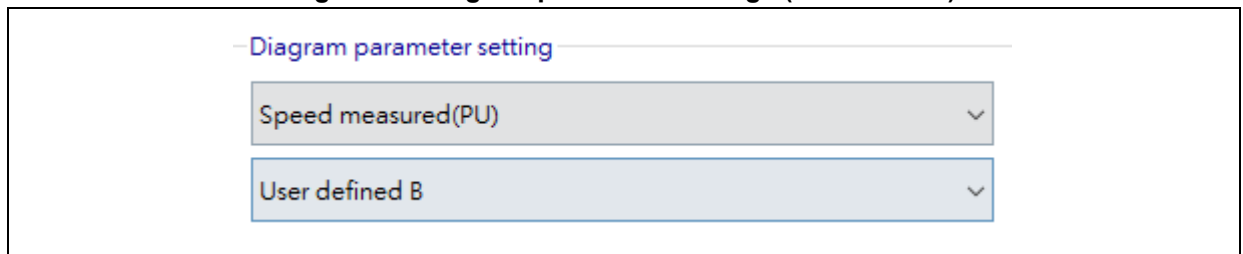
Name	Value
current	0x2000005C ¤t
+ lbus	0x2000005C ¤t
+ labc	0x20000066
+ labc_shunt	0x2000006C
+ lalphabeta	0x20000072
+ lqd	0x20000076
+ lqdref	0x2000007A
q	0
d	0

STEP-7: Align the motor encoder to ensure accurate speed feedback for control stability and enter the speed closed-loop control to prepare for MTPA/MTPV table generation (prior to table creation, connect the motor to a dynamometer and run it at high speed for 10-20 minutes to achieve thermal stabilization).

STEP-8: Follow the table in Figure 49 to identify the optimal I_q and $-I_d$ values that minimize the resultant current " I_s " across different speeds and torques, starting with mid-range conditions (2500 rpm, 2.5 N·m), while monitoring whether the speed reaches the commanded value and increasing the $-I_d$ command if necessary to enhance speed performance.

STEP-9: Observe the speed waveform and I_s waveform (User-defined B) on the UI to identify the minimum resultant current " I_s " that achieves the target speed/torque, which corresponds to the optimal I_q and $-I_d$ values.

Figure 53. Diagram parameter settings (MTPA/MTPV)



STEP-10: After completing the table generation, populate the table indices into the macro definitions in the *motor_control_param.h* header file as shown in Figure 54. The values for MTPA_MTPV_TABLE_ROWS_NUM and MTPA_MTPV_TABLE_COLS_NUM can be determined from Figure 55.

Figure 54. MTPA/MTPV table indices

```
/* MPTA MTPV table parameters for IPMSM use only */
#define MTPA_MTPV_TABLE_MIN_SPEED      (500) /* rpm */
#define MTPA_MTPV_TABLE_MAX_SPEED      (5000) /* rpm */
#define MTPA_MTPV_TABLE_SPEED_STEP     (500) /* rpm */
#define MTPA_MTPV_TABLE_MAX_TORQUE     (5.0f) /* Nm */
#define MTPA_MTPV_TABLE_TORQUE_STEP    (0.5f) /* Nm */
#define MTPA_MTPV_TABLE_ROWS_NUM       (10) /* speed steps */
#define MTPA_MTPV_TABLE_COLS_NUM       (11) /* Torque steps */
```

Figure 55. MTPA/MTPV table row and column count

		COLS	1	2	3	4	5	6	7	8	9	10	11
ROWS	Torque→												
	Speed ↓	0	0.5	1	1.5	2	2.5	3	3.5	4	4.5	5	
1	500												
2	1000												
3	1500												
4	2000												
5	2500												
6	3000												
7	3500												
8	4000												
9	4500												
10	5000												

STEP-11: After table creation, populate the `Iq_table` and `Id_table` values into the `Id_MTPA_LUT[]` and `Iq_MTPA_LUT[]` matrices in the `MTPA_MTPV_table.c` file.

STEP-12: In the `motor_control_param.h` header file, select the IPMSM macro and define `IPM_MTPA_MTPV_CTRL` to enable torque control mode. Inputting torque commands will trigger table-based MTPA/MTPV control.

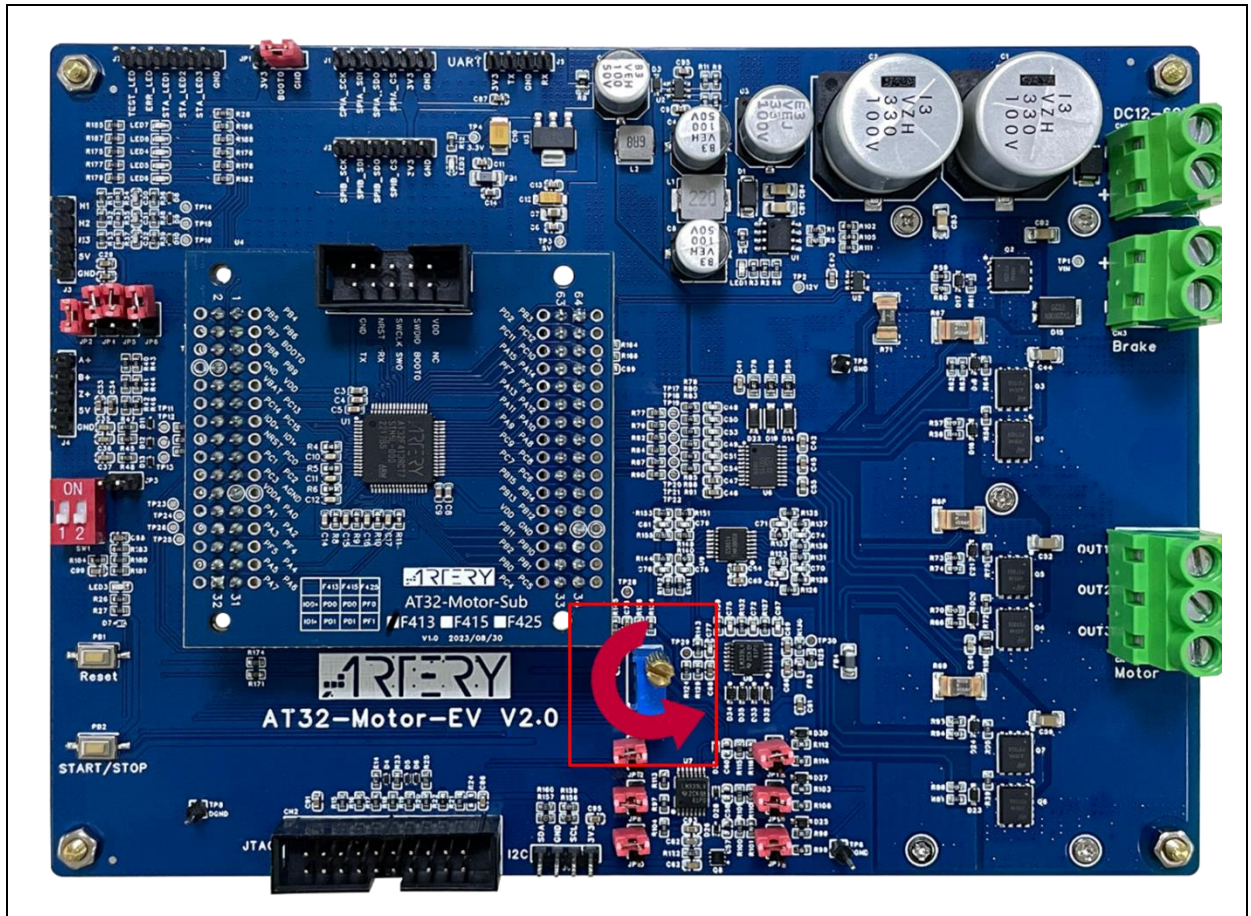
STEP-13: During torque control, verify the Torque reference waveform and the magnitudes of the `Iq`/`Id` current waveforms using a UI monitoring tool to ensure they align with expectations.

STEP-14: When using `IPM_MTPA_MTPV_CTRL`, the speed loop output command is Torque reference (Nm), whereas it is `Iq` reference in SPMSM control. Recalibrate the speed PI control parameters for this mode due to the fundamental difference in command logic.

3.15 External voltage control/software command control

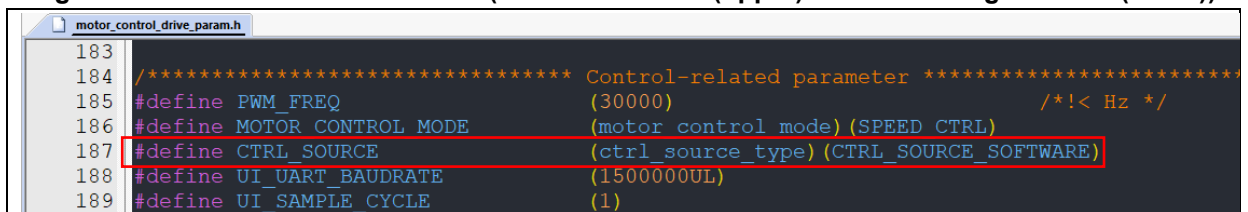
This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 56. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 57) or through PC software (see Figure 58). Control parameters are different for different control modes, such as the target speed (as shown in Figure 58) in speed control mode, or torque reference (as shown in Figure 59) in torque control mode.

Figure 57. Control source definition (software control (upper)/external voltage control (lower))



```

183
184 /***** Control-related parameter *****/
185 #define PWM_FREQ (30000) /*!< Hz */
186 #define MOTOR_CONTROL_MODE (motor_control_mode) (SPEED_CTRL)
187 #define CTRL_SOURCE (ctrl_source_type) (CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE (1500000UL)
189 #define UI_SAMPLE_CYCLE (1)
  
```

1) Select “External voltage control” as a control source

Select “external voltage control” in “Control source”. In this mode, it is possible to adjust speed or torque value by setting external voltage values. The “Target speed” and “Torque reference” area will generate the control speed or torque calculated based on the current voltage.

Note: The “Target speed” and “Torque reference” cannot be modified in “external voltage control” mode.

Figure 58. Speed control mode (external voltage control)

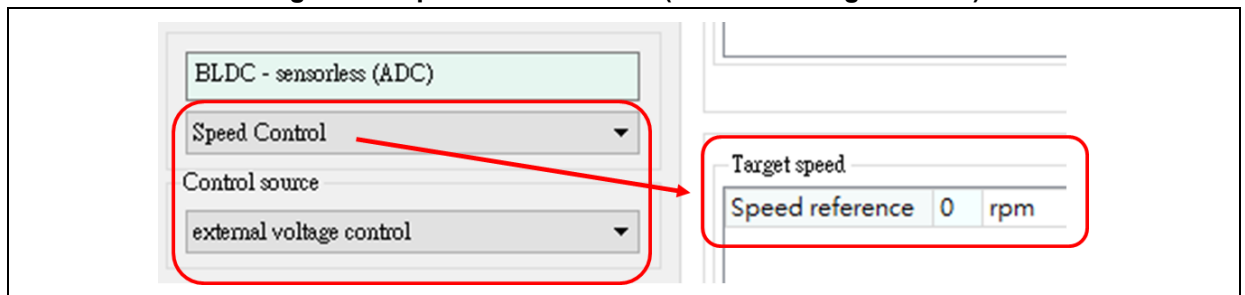
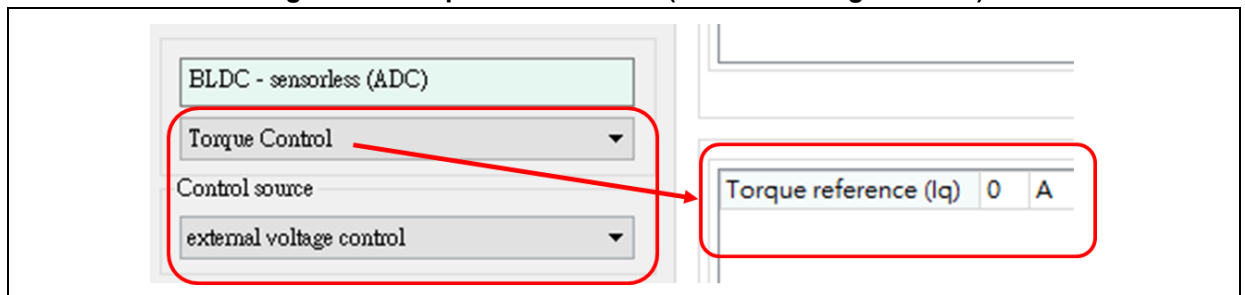


Figure 59. Torque control mode (external voltage control)



2) Select “Software control” as a control source

The software control mode is selected by default, as shown in Figure 60. In this mode, the user can adjust the motor speed and torque by changing the target speed/torque reference. The “message” area will show whether the changing operation is successful or not.

Figure 60. Set target speed in software control mode

The screenshot shows a software control interface. At the top, there's a dropdown menu for 'BLDC - hall'. Below it is a 'Speed Control' dropdown. The 'Control source' dropdown is highlighted with a red box and set to 'software control'. To the right, the 'Target speed' is set to '500 rpm', also highlighted with a red box. Below these are 'Clear' and 'Save' buttons. At the bottom, there's a log table with columns 'Time' and 'Motor'. The log shows three entries: 1. 16:49:32, Set REG 10 = 500:OK; 2. 16:49:26, Set REG 10 = 500:OK; 3. 16:49:22, Set REG 9 = 0:OK. The 'Set REG 10 = 500:OK' entries are highlighted with a red box.

	Time	Motor	Message
1	16:49:32	Set REG 10 = 500:OK	
2	16:49:26	Set REG 10 = 500:OK	
3	16:49:22	Set REG 9 = 0:OK	

3.16 Magnetic encoder offset angle calibration

The magnetic encoder deviation angle is mainly caused by magnetic field distribution offset due to mechanical installation deviations, such as the coaxial error between the motor rotor and the encoder magnetic ring, installation clearance or eccentricity. If the magnetic deviation angle is large and not calibrated, it will directly affect the device control accuracy and stability, such as increased positioning error, dynamic response anomaly, and decreased long-term reliability. An appropriate calibration voltage is required for calibrating magnetic deviation angle. If the calibration voltage is too small, it may cause incorrect angle calibration data, resulting in a worse dynamic response after calibration, whereas excessive calibration voltage may cause overheating damage to the motor. In addition, users need to keep observing the current.

It is necessary to calibrate the magnetic encoder deviation angle at the initial motor debugging or after control board re-programming. Follow the steps below.

STEP-1: Select "Open Loop control"

Figure 61. Select open loop control

The screenshot shows a software control interface. At the top, there's a dropdown menu for 'PMSM - encoder'. Below it, the 'Open Loop control' dropdown is highlighted with a red box and set to 'Open Loop control'.

STEP-2: Go to "Auto tune" to set the "Encoder Calibrate Voltage" (excessive calibration voltage may cause overheating damage to the motor, while a too small calibration voltage may cause incorrect angle calibration data). Click on "Encoder calibration" button, and the motor starts to run in open loop mode. The motor rotates once in both forward and reverse directions, and record the calibration angles.

Figure 62. Magnetic encoder calibration parameters

Encoder Calibration

Encoder Calibrate process state	PROCESS_0_UNDO	
Encoder calibrate Voltage	0.802	V

<
>

Encoder calibration

STEP-3: If the motor fails to run smoothly or it cannot run, fine-tune the “Encoder Calibrate Voltage”, observe the current and check whether the motor runs properly.

STEP-4: Wait for the motor to stop running, and check whether the Encoder Calibrate process state is “PROCESS_5_FINISH”.

STEP-5: After angle calibration is completed, the code stores the deviation lookup table (shortEncoderTable) and encoder offset to the FLASH. Users can start the motor without re-calibration after control board repowering.

Note: If encoder error occurs, it is necessary to check whether the motor’s encoder cable is being connected with the EVB encoder interface. In case of error, it is possible to remove it by clicking on “Fault Ack”.

Note: The magnetic encoder offset angle calibration includes zero-position calibration; therefore, zero-position calibration is not required after encoder calibration.

4 Revision history

Table 25. Document revision history

Date	Version	Revision note
2025.10.09	2.0.0	Initial release.

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.

© 2025 ARTERY Technology – All Rights Reserved